

REPORT

Thiết kế và thực hiện bộ tăng tốc cho mô hình TinyYoLo-Lite bằng phương pháp HLS

Ver 1.0

20/05/2026

	Họ và tên (Full name)	Mã SV (ID)
Thành viên 1 (Member 1)	Nguyễn Hoàng Trung	23021930
Tên/Địa chỉ Repo trên Github hoặc Google Drive	https://github.com/23021930-maker/finalprojectyololite/tree/main	

Abstract (from 5 to 10 lines)

Trong lĩnh vực thị giác máy tính, bài toán phát hiện vật thể đòi hỏi sự cân bằng tối ưu giữa hai yếu tố: độ chính xác và chi phí tính toán. Khung thuật toán YOLO (You Only Look Once), giới thiệu bởi Redmon et al., đã cách mạng hóa lĩnh vực này bằng cách chuyển đổi bài toán phân loại cục bộ thành một bài toán hồi quy đơn nhất.

Tuy nhiên, các mô hình YOLO tiêu chuẩn sở hữu số lượng tham số lớn và mật độ tính toán cao, gây khó khăn khi triển khai trên các thiết bị biên hoặc hệ thống nhúng có tài nguyên hạn chế. Tiny YOLO-lite được phát triển như một biến thể thu gọn nhằm tối ưu hóa tốc độ thực thi bằng cách tinh giản cấu trúc mạng backbone và giảm thiểu các nhánh dự đoán đa quy mô. Báo cáo này đề xuất một số phương pháp hiện bộ tăng tốc cho mô hình Tiny YOLO-lite bằng phương pháp Tổng hợp mức cao – High Level Synthesis.

Keywords

Conv

Relu

Maxpooling

Directive

Optimization

Sigmoid

Detectionhead

Vivado HLS 2018.3

Document History

Version	Time	Revised by	Description
V1.0	21/05/2026	Nguyễn Kiên Hùng	Original Version
V2.0	6/6/2026	Nguyễn Hoàng Trung	Version 1

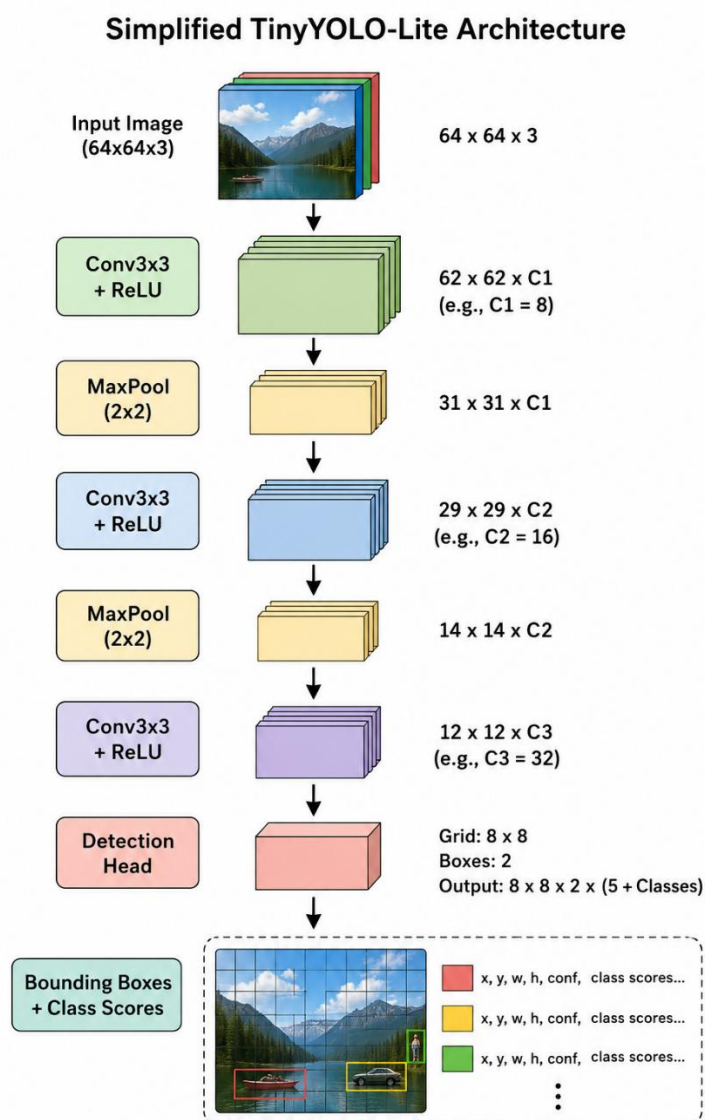
MỤC LỤC

Document History	3
MỤC LỤC	4
1. Giới thiệu.....	5
2. Yêu cầu	6
2.1. <i>Yêu cầu đối với thiết kế:</i>	6
3. Thuật toán.....	7
3.1. <i>Convolution layer</i>	7
3.2. <i>Max Pooling</i>	8
3.3. <i>Detection head</i>	8
4. Mô hình C và testbench	9
4.1. <i>Mô hình C</i>	9
4.2. <i>Xây dựng test bench</i>	20
4.3. <i>Kết quả C simulation</i>	28
4.4. <i>Kết quả C synthesis</i>	30
5. Tối ưu thiết kế để nâng cao hiệu năng.....	30
6. Mô phỏng/thực thi và đánh giá	51
6.1. <i>Kết quả synthesis</i>	51
6.2. <i>Kết quả C/RTL simulation</i>	52
7. Kết quả triển khai phần cứng trên ZynQ	53
8. Kết luận	53
List of Figures	54
List of Code	55

1. Giới thiệu

(Introduction to the motivation, Objectives, and main Contents of the project)

Mục tiêu: Vận dụng các kiến thức, kỹ năng đã được học để thiết kế, mô phỏng và thực thi một mô-đun phần cứng thực hiện chức năng suy luận của mô hình TinyYoLo-Lite. Trong đó, mô hình TinyYoLo-lite bao gồm các lớp cơ bản như nhân chập (Convolution), gộp (MaxPooling), phát hiện (lightweight detection head), tiên đoán hợp bao (Bounding Box predediction) như chỉ ra trong Hình 1.



Hình 1. Cấu trúc phân lớp của mô hình TinyYoLo-lite.

Báo cáo này trình bày chi tiết về cách thiết kế một mô hình C thực hiện chức năng suy luận mô hình TinyYolo-Lite sử dụng phần mềm Vivado HLS 2018.3, với kit ZYNQ-7 ZC702 Evaluation Board. Luồng thủ tục thiết kế tổng quát được tiến hành theo các bước:

B1	Thiết kế thuật toán mô phỏng chức năng suy luận của TinyYolo
B2	Thiết kế Testbench cho thuật toán mô phỏng
B3	Tiến hành C simulation
B4	Tiến hành C synthesis
B5	Tiến hành C/RTL Co-simulation
B6	Debug và đánh giá sử dụng chức năng analysis kiểm tra 3 bước trước đó
B7	Export RTL

Cấu trúc báo cáo:

- ❖ **Phần 1:** Giới thiệu chung
- ❖ **Phần 2:** Yêu cầu thiết kế
- ❖ **Phần 3:** Thuật toán sử dụng
- ❖ **Phần 4:** Nghiên cứu mô hình chưa tối ưu
- ❖ **Phần 5:** Nghiên cứu phương pháp tối ưu
- ❖ **Phần 6:** Đánh giá kết quả tối ưu
- ❖ **Phần 7:** Triển khai mô hình trên kit đánh giá hiệu năng và tài nguyên sử dụng.

2. Yêu cầu

2.1. Yêu cầu đối với thiết kế:

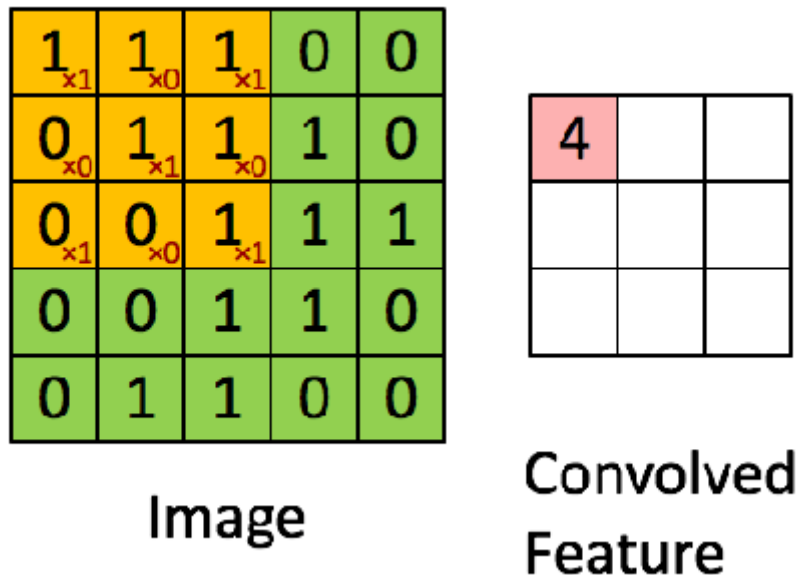
- Xây dựng thuật toán thực hiện từng mô-đun trong mô hình TinyYoLo-Lite.
- Xây dựng bản mô tả mức cao cho từng mô-đun trong mô hình TinyYoLo-Lite bằng ngôn ngữ C, từ đó tạo bản mô tả top-level cho mô hình TinyYoLo-Lite
- Xây dựng testbench để kiểm chứng mô hình TinyYoLo-Lite bằng C simulation
- Tổng hợp bản mô tả C thành bản mô tả RTL bằng VHDL/Verilog bằng Vivado HLS
- Mô phỏng C/RTL co-simulation để kiểm chứng và phân tích hoạt động của phần cứng sau tổng hợp HLS
- Tối ưu mô hình C và thêm vào các dẫn hướng để cải thiện hiệu năng của thiết kế
- Thực hiện thiết kế trên ZynQ-7000 để đánh giá lượng tài nguyên sử dụng, hiệu năng.

3. Thuật toán

- Việc thực hiện chức năng suy luận của mô hình TinyYoLo-Lite được chia ra thành nhiều bước và có thể tóm gọn trong các bước chính: Convolution, MaxPooling, Detection head

3.1. Convolution layer

- Convolution (lớp tích chập) là lớp đầu tiên trích xuất các đặc tính từ hình ảnh. Tham số lớp này bao gồm một tập hợp các bộ lọc có thể học được. Các bộ lọc đều nhỏ có kích cỡ 3x3 và có độ sâu bằng với độ sâu của đầu vào đầu vào. Bằng cách trượt dần bộ lọc theo chiều ngang và dọc trên ảnh, chúng ta thu được một Feature Map chứa các đặc trưng được trích xuất từ trên hình ảnh đầu vào.



Hình 2. Ví dụ minh họa phép tính tích chập

Công thức tính toán:

$$Y(o, r, c) = B(o) + \sum_{i=0}^{C_{in}-1} \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} X(i, r+m, c+n) \cdot W(o, i, m, n)$$

Trong đó:

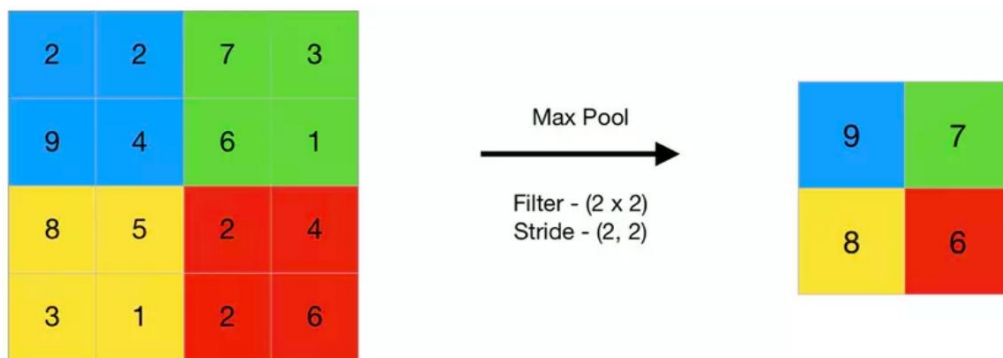
- X : tensor đầu vào
- Y : tensor đầu ra
- W : trọng số (kernel/filter)

- B : bias
- i : chỉ số kênh đầu vào (input channel)
- o : chỉ số kênh đầu ra (output channel)
- r, c : tọa độ điểm ảnh đầu ra
- m, n : tọa độ trong kernel
- K : kích thước kernel (ở đây $K = 3$)

Mã trận ảnh đầu ra sẽ được đi qua Relu layer cho phép huấn luyện nhanh hơn và hiệu quả hơn bằng cách quy đổi các giá trị âm về 0 và giữ nguyên các giá trị dương.

3.2. Max Pooling

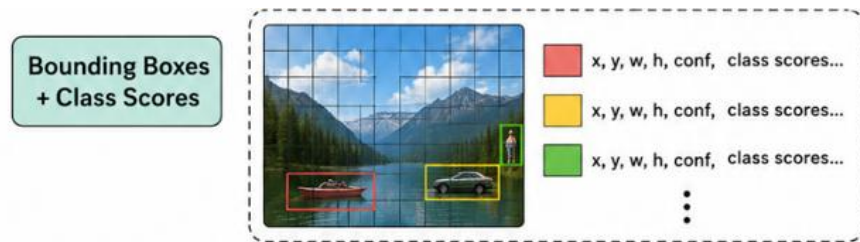
- Pooling layer thường được dùng giữa các convolutional layer, để giảm kích thước dữ liệu nhưng vẫn giữ được các thuộc tính quan trọng. Kích thước dữ liệu giảm giúp giảm việc tính toán trong model. Trong quá trình này, quy tắc về stride và padding áp dụng như phép tính convolution trên ảnh.



Hình 3. Ví dụ minh họa max pooling

3.3. Detection head

- Detection head đóng vai trò là lớp ra quyết định cuối cùng trong kiến trúc mạng thần kinh phát hiện đối tượng. Trong khi các lớp trước của mô hình chịu trách nhiệm hiểu hình dạng, kết cấu và đặc trưng trong ảnh, thì detection head là thành phần cụ thể diễn giải thông tin này để dự đoán chính xác đối tượng nào đang hiện diện và vị trí của chúng. Nó chuyển đổi dữ liệu trừu tượng, cấp cao được tạo ra bởi bộ trích xuất đặc trưng thành kết quả thực thi được, xuất ra một tập hợp các bounding boxes bao quanh các đối tượng được xác định cùng với nhãn lớp và confidence scores tương ứng.



Hình 4: Ví dụ minh họa detection head

Detection Head là nơi đưa ra các con số dự đoán thô (Raw outputs), để biến đổi các con số thô đó thành các giá trị có ý nghĩa thực tế thì cần dùng Activation Function chính là bộ lọc quyết định (như tọa độ pixel, tỷ lệ phần trăm xác suất).

4. Mô hình C và testbench

4.1. Mô hình C

C Source files:

topp.h
<pre> #include <stdio.h> #define IMG_H 64 #define IMG_W 64 #define IMG_C 3 #define C1 8 #define C2 16 #define C3 32 #define NUM_CLASSES 3 #define NUM_BOXES 2 #define OUT_CH (NUM_BOXES * (5 + NUM_CLASSES)) </pre>

Code 1: file nguồn thuật toán

C code file

topp.c

```
#include "topp.h"
#include <math.h>

float W1[C1][3][3][3];
float B1[C1];

float W2[C2][C1][3][3];
float B2[C2];

float W3[C3][C2][3][3];
float B3[C3];

float W4[C3][C3][3][3];
float B4[C3];

float W5[C3][C3][3][3];
float B5[C3];

float WH[OUT_CH][C3][1][1];
float BH[OUT_CH];

void conv1(
    float in[64][64][3],
    float out[62][62][C1],
    float kernel[C1][3][3][3],
    float bias[C1])
{
    for(int oc=0; oc<C1; oc++)
```

```

{
    for(int r=0; r<62; r++)
    {
        for(int c=0; c<62; c++)
        {
            float sum = bias[oc];

            for(int ic=0; ic<3; ic++)
            {
                for(int kr=0; kr<3; kr++)
                {
                    for(int kc=0; kc<3; kc++)
                    {
                        sum += in[r+kr][c+kc][ic] * kernel[oc][kr][kc][ic];
                    }
                }
            }

            if(sum < 0) sum = 0;
            out[r][c][oc] = sum;
        }
    }
}

```

```

void maxpool1(
    float in[62][62][C1],

```

```

float out[31][31][C1])
{
    for(int ch=0; ch<C1; ch++)
    {
        for(int r=0; r<31; r++)
        {
            for(int c=0; c<31; c++)
            {
                float maxv = in[2*r][2*c][ch];

                for(int i=0; i<2; i++)
                {
                    for(int j=0; j<2; j++)
                    {
                        float v = in[2*r+i][2*c+j][ch];
                        if(v > maxv) maxv = v;
                    }
                }
                out[r][c][ch] = maxv;
            }
        }
    }
}

void conv2(
    float in[31][31][C1],
    float out[29][29][C2],
    float kernel[C2][C1][3][3],
    float bias[C2])

```

```

{

for(int oc=0; oc<C2; oc++)
{
    for(int r=0; r<29; r++)
    {
        for(int c=0; c<29; c++)
        {
            float sum = bias[oc];

            for(int ic=0; ic<C1; ic++)
            {
                for(int kr=0; kr<3; kr++)
                {
                    for(int kc=0; kc<3; kc++)
                    {
                        sum += in[r+kr][c+kc][ic] * kernel[oc][ic][kr][kc];
                    }
                }
            }

            if(sum < 0) sum = 0;
            out[r][c][oc] = sum;
        }
    }
}

```

```

void maxpool2(

```

```

float in[29][29][C2],
float out[14][14][C2])
{

for(int ch=0; ch<C2; ch++)
{
    for(int r=0; r<14; r++)
    {
        for(int c=0; c<14; c++)
        {
            float maxv = in[2*r][2*c][ch];

            for(int i=0; i<2; i++)
            {
                for(int j=0; j<2; j++)
                {
                    float v = in[2*r+i][2*c+j][ch];
                    if(v > maxv) maxv = v;
                }
            }
            out[r][c][ch] = maxv;
        }
    }
}

void conv3(
    float in[14][14][C2],
    float out[12][12][C3],

```

```

float kernel[C3][C2][3][3],
float bias[C3])
{

for(int oc=0; oc<C3; oc++)
{
for(int r=0; r<12; r++)
{
for(int c=0; c<12; c++)
{
float sum = bias[oc];

for(int ic=0; ic<C2; ic++)
{
for(int kr=0; kr<3; kr++)
{
for(int kc=0; kc<3; kc++)
{
sum += in[r+kr][c+kc][ic] * kernel[oc][ic][kr][kc];
}
}
}
}

if(sum < 0) sum = 0;
out[r][c][oc] = sum;
}
}
}
}

```

```

void conv4(
    float in[12][12][C3],
    float out[10][10][C3],
    float kernel[C3][C3][3][3],
    float bias[C3])
{
    for(int oc=0; oc<C3; oc++)
    {
        for(int r=0; r<10; r++)
        {
            for(int c=0; c<10; c++)
            {
                float sum = bias[oc];

                for(int ic=0; ic<C3; ic++)
                {
                    for(int kr=0; kr<3; kr++)
                    {
                        for(int kc=0; kc<3; kc++)
                        {
                            sum += in[r+kr][c+kc][ic] * kernel[oc][ic][kr][kc];
                        }
                    }
                }

                if(sum < 0) sum = 0;
                out[r][c][oc] = sum;
            }
        }
    }
}

```



```

    }
}

void conv5(
    float in[10][10][C3],
    float out[8][8][C3],
    float kernel[C3][C3][3][3],
    float bias[C3])
{
    for(int oc=0; oc<C3; oc++)
    {
        for(int r=0; r<8; r++)
        {
            for(int c=0; c<8; c++)
            {
                float sum = bias[oc];

                for(int ic=0; ic<C3; ic++)
                {
                    for(int kr=0; kr<3; kr++)
                    {
                        for(int kc=0; kc<3; kc++)
                        {
                            sum += in[r+kr][c+kc][ic] * kernel[oc][ic][kr][kc];
                        }
                    }
                }
                out[r][c][oc] = sum;
            }
        }
    }
}

```

```

    }
}
}

float sigmoid(float x)
{
    return 1.0f / (1.0f + expf(-x));
}

void detection_head(
    float in[8][8][C3],
    float out[8][8][OUT_CH],
    float kernel[OUT_CH][C3][1][1],
    float bias[OUT_CH])
{
    for(int oc=0; oc<OUT_CH; oc++)
    {
        for(int r=0; r<8; r++)
        {
            for(int c=0; c<8; c++)
            {
                float sum = bias[oc];
                for(int ic=0; ic<C3; ic++)
                {
                    sum += in[r][c][ic] * kernel[oc][ic][0][0];
                }
            }
        }
    }
}

```

```

        out[r][c][oc] = sigmoid(sum);
    }
}
}
}

```

```

void topp(
    float image[64][64][3],
    float output[8][8][OUT_CH]
)
{
    static float f1[62][62][C1];
    static float p1[31][31][C1];
    static float f2[29][29][C2];
    static float p2[14][14][C2];
    static float f3[12][12][C3];
    static float f4[10][10][C3];
    static float f5[8][8][C3];

    conv1(image, f1, W1, B1);
    maxpool1(f1, p1);
    conv2(p1, f2, W2, B2);
    maxpool2(f2, p2);
    conv3(p2, f3, W3, B3);
    conv4(f3, f4, W4, B4);
    conv5(f4, f5, W5, B5);
    detection_head(f5, output, WH, BH);
}

```

Code 2: thuật toán TinyYolo-lite cơ bản

+ topp.h chứa định nghĩa kích thước của ma trận đầu vào (IMG_H 64, IMG_W, IMG_C), số feature map của các lớp convolution (C1, C2, C3), số lớp cần phân loại (NUM_CLASSES), số bounding box dự đoán cho mỗi ô (NUM_BOXES), số kênh output cuối (OUT_CH)

+ topp.c chứa phần code thực hiện lần lượt các bước tính toán để tạo ra được mô hình suy luận TinyYolo lite:

Conv => Relu => MaxPooling => Conv => Relu => MaxPooling => Conv => Relu => Conv
=> Relu => Conv => Detection head

Đây là một cách triển khai đơn giản, mang tính chất trực quan ban đầu, chưa sử dụng bất kỳ cấu trúc dữ liệu chuyên biệt nào để tối ưu thuật toán. Đồng thời, cũng chưa áp dụng các chỉ thị dẫn hướng nhằm cải thiện các thông số hiệu năng như throughput, latency.

4.2. Xây dựng test bench

Code testbench

tb_top.c
<pre>#include <stdio.h> #include "topp.h" /*===== =====*/ extern float W1[C1][3][3][3]; extern float B1[C1]; extern float W2[C2][C1][3][3]; extern float B2[C2]; extern float W3[C3][C2][3][3]; extern float B3[C3]; extern float W4[C3][C3][3][3]; extern float B4[C3];</pre>

```

extern float W5[C3][C3][3][3];
extern float B5[C3];

extern float WH[OUT_CH][C3][1][1];
extern float BH[OUT_CH];

/*=====

=====*/

static float tb_image[64][64][3];
static float tb_output[8][8][OUT_CH];

/*=====

=====*/

void tb_init_image()
{
    int r,c,ch;

    for(r=0;r<64;r++)
    {
        for(c=0;c<64;c++)
        {
            for(ch=0;ch<3;ch++)
            {
                tb_image[r][c][ch] = 0.0f;
            }
        }
    }
}

```

```

for(r=22;r<42;r++)
{
    for(c=22;c<42;c++)
    {
        for(ch=0;ch<3;ch++)
        {
            tb_image[r][c][ch] = 1.0f;
        }
    }
}

/*=====
Weight = 1
=====*/

void tb_init_weights()
{
    int oc, ic, r, c;

    /* Conv1 : average filter */
    for(oc=0; oc<C1; oc++)
    {
        B1[oc] = 0.0f;

        for(r=0; r<3; r++)
        {
            for(c=0; c<3; c++)
            {
                for(ic=0; ic<3; ic++)

```

```

        {
            W1[oc][r][c][ic] = 1.0f/27.0f;
        }
    }
}

```

```

/* Conv2 : identity */

```

```

for(oc=0; oc<C2; oc++)

```

```

{
    B2[oc] = 0.0f;

```

```

    for(ic=0; ic<C1; ic++)

```

```

    {
        for(r=0; r<3; r++)

```

```

        {
            for(c=0; c<3; c++)

```

```

            {
                W2[oc][ic][r][c] = 0.0f;

```

```

                if(r==1 && c==1)

```

```

                    W2[oc][ic][r][c] = 1.0f;

```

```

            }

```

```

        }

```

```

    }

```

```

}

```

```

/* Conv3 : identity */

```

```

for(oc=0; oc<C3; oc++)

```

```

{

```

```

    B3[oc] = 0.0f;

    for(ic=0; ic<C2; ic++)
    {
        for(r=0; r<3; r++)
        {
            for(c=0; c<3; c++)
            {
                W3[oc][ic][r][c] = 0.0f;

                if(r==1 && c==1)
                    W3[oc][ic][r][c] = 1.0f;
            }
        }
    }
}

/* Conv4 : identity */
for(oc=0; oc<C3; oc++)
{
    B4[oc] = 0.0f;

    for(ic=0; ic<C3; ic++)
    {
        for(r=0; r<3; r++)
        {
            for(c=0; c<3; c++)
            {
                W4[oc][ic][r][c] = 0.0f;

```



```

        if(r==1 && c==1)
            W4[oc][ic][r][c] = 1.0f;
    }
}
}
}

```

```

/* Conv5 : identity */

```

```

for(oc=0; oc<C3; oc++)
{
    B5[oc] = 0.0f;

    for(ic=0; ic<C3; ic++)
    {
        for(r=0; r<3; r++)
        {
            for(c=0; c<3; c++)
            {
                W5[oc][ic][r][c] = 0.0f;

                if(r==1 && c==1)
                    W5[oc][ic][r][c] = 1.0f;
            }
        }
    }
}

```

```

/* Detection Head */

```

```

for(oc=0; oc<OUT_CH; oc++)
{

```

```

    BH[oc] = 0.0f;

    for(ic=0; ic<C3; ic++)
    {
        WH[oc][ic][0][0] = 1.0f / (float)C3;
    }
}

/*=====
Main
=====*/

int main()
{
    int r,c;

    printf("Init weights...\n");
    tb_init_weights();

    printf("Init image...\n");
    tb_init_image();

    printf("Run CNN...\n");
    topp(tb_image, tb_output);

    printf("\nOutput channel 0\n\n");

    for(r=0;r<8;r++)
    {
        for(c=0;c<8;c++)
        {

```

```

        printf("%6.3f ", tb_output[r][c][0]);
    }
    printf("\n");
}

```

```

printf("\nDetected cells:\n\n");

```

```

for(int gr=0; gr<8; gr++)
{
    for(int gc=0; gc<8; gc++)
    {
        float conf = tb_output[gr][gc][4];

        if(conf > 0.5f)
        {
            printf(
                "Cell(%d,%d): "
                "x=%.2f y=%.2f "
                "w=%.2f h=%.2f "
                "conf=%.2f class=%.2f\n",

                gr,
                gc,

                tb_output[gr][gc][0],
                tb_output[gr][gc][1],
                tb_output[gr][gc][2],
                tb_output[gr][gc][3],
                tb_output[gr][gc][4],

```

```

        tb_output[gr][gc][5]
    );
}
}
}

return 0;
}

```

Code 3: testbench thuật toán TinyYolo-lite cơ bản

Test bench thực hiện các việc sau:

- Tạo ảnh đầu vào giả lập là một khối trắng có nền đen ở giữa
- Gán trọng số thủ công:
 - +Conv1: Ma trận có tất cả trọng số là 1/27
 - +Conv2, 3, 4, 5: Ma trận toàn 0 chứa 1 ở giữa
- Chạy CNN.
- In kết quả đầu ra

4.3. Kết quả C simulation

In ra channel 0 để đánh giá:

```

19
20 Output channel 0
21
22 0.500 0.500 0.500 0.500 0.500 0.500 0.500 0.500
23 0.500 1.000 1.000 1.000 1.000 1.000 1.000 0.500
24 0.500 1.000 1.000 1.000 1.000 1.000 1.000 0.500
25 0.500 1.000 1.000 1.000 1.000 1.000 1.000 0.500
26 0.500 1.000 1.000 1.000 1.000 1.000 1.000 0.500
27 0.500 1.000 1.000 1.000 1.000 1.000 1.000 0.500
28 0.500 1.000 1.000 1.000 1.000 1.000 1.000 0.500
29 0.500 0.500 0.500 0.500 0.500 0.500 0.500 0.500
30

```

Hình 5: Đầu ra channel 0 test bench

```

31 Detected cells:
32
33 Cell(1,1): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
34 Cell(1,2): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
35 Cell(1,3): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
36 Cell(1,4): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
37 Cell(1,5): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
38 Cell(1,6): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
39 Cell(2,1): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
40 Cell(2,2): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
41 Cell(2,3): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
42 Cell(2,4): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
43 Cell(2,5): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
44 Cell(2,6): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
45 Cell(3,1): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
46 Cell(3,2): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
47 Cell(3,3): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
48 Cell(3,4): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
49 Cell(3,5): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
50 Cell(3,6): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
51 Cell(4,1): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
52 Cell(4,2): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
53 Cell(4,3): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
54 Cell(4,4): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
55 Cell(4,5): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
56 Cell(4,6): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
57 Cell(5,1): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
58 Cell(5,2): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
59 Cell(5,3): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
60 Cell(5,4): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
61 Cell(5,5): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
62 Cell(5,6): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
63 Cell(6,1): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
64 Cell(6,2): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
65 Cell(6,3): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
66 Cell(6,4): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
67 Cell(6,5): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
68 Cell(6,6): x=1.00 y=1.00 w=1.00 h=1.00 conf=1.00 class=1.00
69 INFO: [SIM 1] CSim done with 0 errors.
70 INFO: [SIM 3] ***** CSIM finish *****
71

```

Hình 6: Đầu ra bounding box + class score test bench

=> Các giá trị x, y, w, h lần lượt thể hiện tọa độ tâm vật thể, và độ cao chiều rộng của bounding box, kết quả cho thấy thuật toán đưa ra được tọa độ khối vuông với confidence=1 tức tỉ lệ phát hiện là 100%, tuy nhiên do tất cả channel đều dùng 1 weight nên kết quả chỉ mang tính tham khảo, không có thực sự đánh giá được khả năng phát hiện vật thể của mô hình. Ngoài ra do

không thực sự train các trọng số nên tọa độ tâm, độ rộng bounding box và class đều không mang ý nghĩa thực tiễn

4.4. Kết quả C synthesis

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	8.677	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
48873231	48873231	48873231	48873231	none

Detail

Instance

Instance	Module	Latency		Interval		Type
		min	max	min	max	
grp_detection_head_fu_40	detection_head	393505	393505	393505	393505	none
grp_conv2_fu_48	conv2	11559665	11559665	11559665	11559665	none
grp_conv1_fu_56	conv1	10794961	10794961	10794961	10794961	none
grp_conv4_fu_64	conv4	10967105	10967105	10967105	10967105	none
grp_conv3_fu_72	conv3	7903553	7903553	7903553	7903553	none
grp_conv5_fu_80	conv5	7017025	7017025	7017025	7017025	none
grp_maxpool1_fu_88	maxpool1	177337	177337	177337	177337	none
grp_maxpool2_fu_96	maxpool2	60065	60065	60065	60065	none

Loop

N/A

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	-	-	-
Expression	-	-	-	-
FIFO	-	-	-	-
Instance	-	37	5557	11476
Memory	148	-	0	0
Multiplexer	-	-	-	358
Register	-	-	24	-
Total	148	37	5581	11834
Available	280	220	106400	53200
Utilization (%)	52	16	5	22

Hình 7: Kết quả synthesis mô hình suy luận cơ bản

Nhận xét: Kết quả synthesis cho thấy giá trị clock đạt được thỏa mãn được mục tiêu đề ra, tuy nhiên latency và interval lại có giá trị vô cùng lớn, điều này xảy ra do không áp dụng dẫn hướng nên mạch thực hiện tuần tự tất cả các khối lệnh vậy nên khi thực hiện các hàm có nhiều vòng for lồng nhau như conv hay maxpooling thì việc duyệt qua các vòng for trở nên tốn thời gian.

5. Tối ưu thiết kế để nâng cao hiệu năng

Vấn đề của thuật toán ban đầu:

Trong mỗi lần thực hiện việc nhân chập, việc dịch kernel đi 1 stride thì chương trình lại phải đọc lại 9 pixel đó mà số lượng cổng đọc có hạn nên việc phải đợi cổng đọc thì gây trễ rất lớn

Tối ưu thuật toán:

Việc tối ưu thuật toán được thực hiện bằng cách áp dụng Line and Window Buffers với mục tiêu làm cho việc truy cập dữ liệu từ ma trận đầu vào là ít nhất có thể, tức mỗi một phần tử chỉ cần phải truy cập duy nhất một lần

- Line buffer:

Line buffer được sử dụng để đệm nhiều dòng của một hình ảnh đầy đủ, và cụ thể, ở đây nó được thiết kế để đệm $FILTER_V_SIZE - 1$ dòng hình ảnh. Trong đó $FILTER_V_SIZE$ là chiều cao của bộ lọc tích chập. Tổng số pixel được lưu trữ bởi bộ đệm dòng là $(FILTER_V_SIZE - 1)$. Line buffer giải quyết vấn đề bằng cách thay vì lưu toàn bộ ảnh trong RAM rồi đọc đi đọc lại, ta giữ các hàng gần nhất để tái sử dụng.

- Window Buffers:

Window Buffers chứa $\text{FILTER_V_SIZE} * \text{FILTER_H_SIZE}$ pixel. Trong đó FILTER_H_SIZE là chiều rộng của bộ lọc tích chập. Thao tác lọc tích chập 2 chiều bao gồm việc căn giữa mặt nạ lọc (hệ số lọc) vào chỉ số của pixel đầu ra và tính tổng tích (SOP). Window Buffers giải quyết vấn đề bằng cách thay vì mỗi lần dịch lại lưu lại toàn bộ pixel trong RAM rồi đọc đi đọc lại, ta chỉ cần nạp thêm FILTER_H_SIZE pixel mới mỗi lần dịch.

Tối ưu bằng cách gán các dẫn hướng:

Việc gán các dẫn hướng nhằm giúp hướng dẫn bộ tổng hợp phần cứng tối ưu kiến trúc RTL sinh ra. Nếu không có directive, HLS thường tạo phần cứng theo cách an toàn nhưng chưa tối ưu về tốc độ hoặc tài nguyên.

#pragma HLS ARRAY_PARTITION variable=X complete dim=X

Dẫn hướng cho phép chia một mảng thành các phần tử nhỏ hơn. Kết quả là RTL có nhiều bộ nhớ nhỏ hoặc nhiều thanh ghi thay vì một bộ nhớ lớn:

- + Tăng hiệu quả số lượng cổng đọc và ghi cho bộ nhớ.
- + Có khả năng cải thiện thông lượng của thiết kế.
- + Yêu cầu nhiều thể hiện bộ nhớ hoặc thanh ghi hơn.

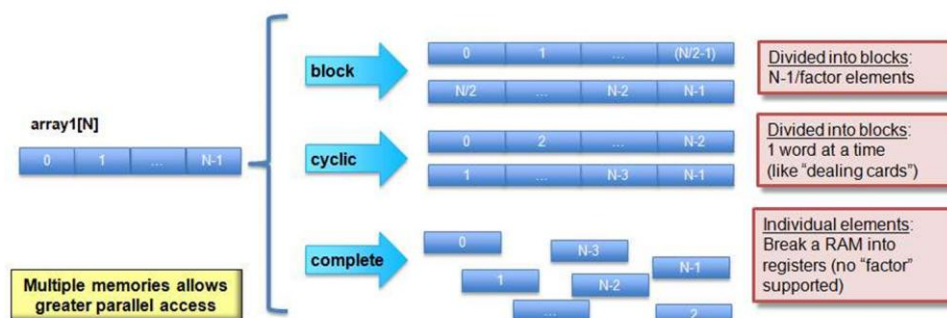
- **Lựa chọn complete:** Phân vùng hoàn chỉnh phân tách mảng thành các phần tử riêng lẻ. Đối với mảng một chiều, điều này tương ứng với việc phân giải bộ nhớ thành các thanh ghi riêng lẻ.

- **Lựa chọn cyclic:** Phân vùng tuần hoàn tạo ra các mảng nhỏ hơn bằng cách xen kẽ các phần tử từ mảng ban đầu. Mảng được phân vùng theo chu kỳ bằng cách đưa một phần tử vào mỗi mảng mới trước khi quay lại mảng đầu tiên để lặp lại chu kỳ cho đến khi mảng được phân vùng hoàn toàn.

- **Lựa chọn dim:** Chỉ định chiều nào của mảng đa chiều cần phân vùng. Giá trị mặc định là 1. Được chỉ định dưới dạng số nguyên từ 0 đến $\langle N \rangle$, đối với mảng có $\langle N \rangle$ chiều:

Nếu sử dụng giá trị 0, tất cả các chiều của mảng đa chiều sẽ được phân vùng với các tùy chọn kiểu và hệ số được chỉ định.

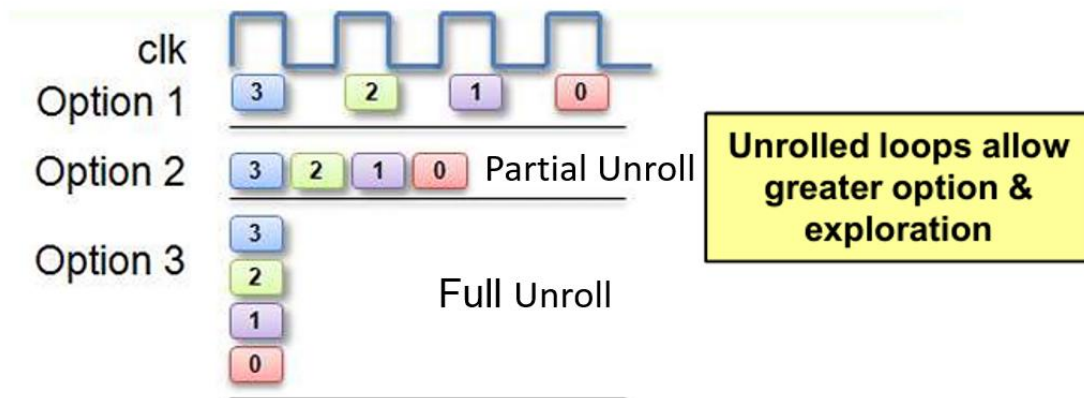
Bất kỳ giá trị nào khác 0 chỉ phân vùng chiều được chỉ định. Ví dụ, nếu sử dụng giá trị 1, chỉ chiều đầu tiên được phân vùng.



Hình 8: Mô tả cách hoạt động của dẫn hướng array_partition

#pragma HLS UNROLL

Chỉ thị UNROLL biến đổi vòng lặp bằng cách tạo ra nhiều bản sao của thân vòng lặp trong thiết kế RTL, cho phép một số hoặc tất cả các lần lặp của vòng lặp diễn ra song song.

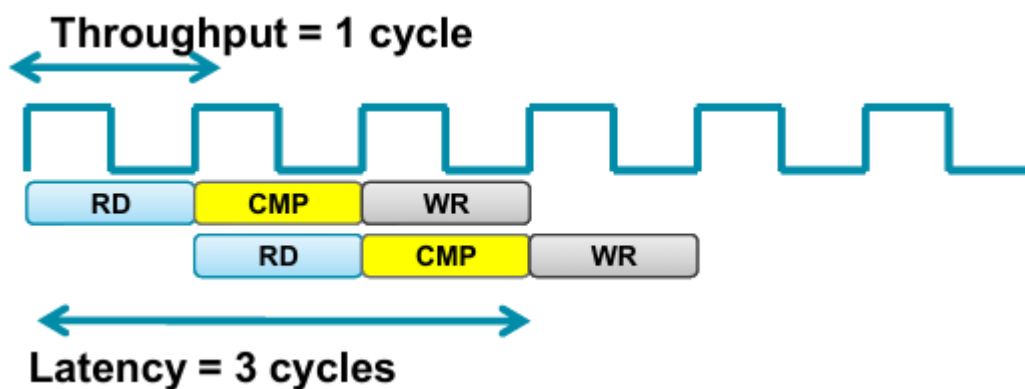


Hình 9: Mô tả cách hoạt động của dẫn hướng unroll

#pragma HLS PIPELINE II=1

Chỉ thị PIPELINE giảm khoảng thời gian khởi tạo (II) cho một hàm hoặc vòng lặp bằng cách cho phép thực thi đồng thời các thao tác. Một hàm hoặc vòng lặp đường ống có thể xử lý các đầu vào mới sau mỗi $\langle N \rangle$ chu kỳ xung nhịp, trong đó $\langle N \rangle$ là II của vòng lặp hoặc hàm. $II=1$ xử lý một đầu vào mới sau mỗi chu kỳ xung nhịp.

Theo mặc định, với chỉ thị hoặc lệnh PIPELINE sẽ tạo ra II tối thiểu cho thiết kế theo ràng buộc chu kỳ xung nhịp được chỉ định. Nếu công cụ không thể tạo ra một thiết kế với II được chỉ định, nó sẽ đưa ra cảnh báo và tạo ra một thiết kế với II thấp nhất có thể.



Hình 10: Mô tả cách hoạt động của dẫn hướng pipeline

#pragma HLS INLINE

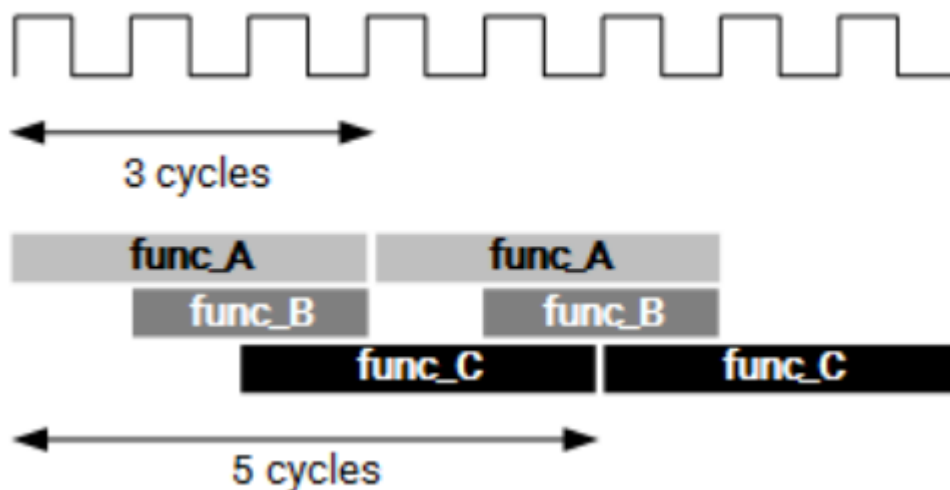
Loại bỏ một hàm như một thực thể riêng biệt trong hệ thống phân cấp. Sau khi inline, hàm đó được hợp nhất vào hàm gọi và không còn xuất hiện như một cấp độ phân cấp riêng biệt trong mã RTL nữa.

Trong một số trường hợp, việc nội tuyến một hàm cho phép các thao tác bên trong hàm được chia sẻ và tối ưu hóa hiệu quả hơn với hàm gọi. Tuy nhiên, một hàm nội tuyến không thể được chia sẻ hoặc tái sử dụng, vì vậy nếu hàm cha gọi hàm nội tuyến nhiều lần, điều này có thể làm tăng diện tích cần thiết để triển khai RTL.

#pragma HLS DATAFLOW

Cho phép xử lý song song ở cấp độ tác vụ, cho phép các hàm và vòng lặp chồng chéo hoạt động, tăng tính đồng thời của triển khai RTL và tăng thông lượng tổng thể của thiết kế.

Tất cả các thao tác được thực hiện tuần tự trong mô tả C. Trong trường hợp không có bất kỳ chỉ thị nào giới hạn tài nguyên, công cụ tìm cách giảm thiểu độ trễ và cải thiện tính đồng thời. Tuy nhiên, sự phụ thuộc dữ liệu có thể hạn chế điều này. Ví dụ, các hàm hoặc vòng lặp truy cập mảng phải hoàn thành tất cả các truy cập đọc/ghi vào mảng trước khi chúng kết thúc. Điều này ngăn hàm hoặc vòng lặp tiếp theo sử dụng dữ liệu bắt đầu hoạt động. Tối ưu hóa DATAFLOW cho phép các thao tác trong một hàm hoặc vòng lặp bắt đầu hoạt động trước khi hàm hoặc vòng lặp trước đó hoàn thành tất cả các thao tác của nó.



Hình 11: Mô tả cách hoạt động của dẫn hướng dataflow

DATAFLOW được chỉnh ở channel mặc định là FIFO. FIFO (First-In, First-Out) là một bộ đệm phân cứng hoạt động theo nguyên lý "vào trước, ra trước".

FIFO Depth chính là sức chứa của bộ đệm đó. Nó quyết định bộ đệm có thể chứa tối đa bao nhiêu phần tử dữ liệu cùng một lúc trước khi bị đầy. Việc đặt depth là 16 nhằm tránh hiện tượng nghẽn:

Ví dụ để tính toán được một ô Kernel 3x3 hàm conv bắt buộc phải tích lũy đủ dữ liệu của 2 hàng đầu tiên cộng thêm 3 pixel của hàng thứ 3. Khi tầng conv1 bắt đầu chạy, nó liên tục nạp dữ liệu vào và đẩy kết quả ra. Tuy nhiên, tầng conv2 phía sau nhận được dữ liệu từ conv1 nhưng chưa thể tính toán ngay được vì nó còn phải đợi buffer cho đủ 2 hàng dữ liệu vào Line Buffer của riêng nó. Tầng conv2 chưa chịu đọc, khiến bộ đệm FIFO giữa conv1 và conv2 bị đầy ngay lập tức. Khi FIFO đầy, tầng conv1 bị ép phải dừng lại không được tính tiếp. Cả hệ thống bị nghẽn

Code sau tối ưu

topp.c
<pre>#include "topp.h" #include <math.h> float W1[C1][3][3][3]; float B1[C1]; float W2[C2][C1][3][3]; float B2[C2]; float W3[C3][C2][3][3]; float B3[C3]; float W4[C3][C3][3][3]; float B4[C3]; float W5[C3][C3][3][3]; float B5[C3]; float WH[OUT_CH][C3][1][1];</pre>

```

float BH[OUT_CH];

/* =====

===== */

void conv1(
    float in[64][64][3],
    float out[62][62][C1],
    float kernel[C1][3][3][3],
    float bias[C1])
{
    static float linbu[2][64][3];
    float win_buf[3][3][3];
    int r, c, ic, oc, kr, kc;
    float val0, val1, val2, sum;

    #pragma HLS ARRAY_PARTITION variable=linbu complete dim=1
    #pragma HLS ARRAY_PARTITION variable=linbu complete dim=3
    #pragma HLS ARRAY_PARTITION variable=win_buf complete dim=0
    for(r = 0; r < 64; r++) {
        for(c = 0; c < 64; c++) {

            for(ic = 0; ic < 3; ic++) {
                #pragma HLS UNROLL
                win_buf[0][0][ic] = win_buf[0][1][ic];
                win_buf[0][1][ic] = win_buf[0][2][ic];
                win_buf[1][0][ic] = win_buf[1][1][ic];
                win_buf[1][1][ic] = win_buf[1][2][ic];
                win_buf[2][0][ic] = win_buf[2][1][ic];
                win_buf[2][1][ic] = win_buf[2][2][ic];
            }
        }
    }
}

```

```

    val0 = linbu[0][c][ic];
    val1 = linbu[1][c][ic];
    val2 = in[r][c][ic];

    linbu[0][c][ic] = val1;
    linbu[1][c][ic] = val2;

    win_buf[0][2][ic] = val0;
    win_buf[1][2][ic] = val1;
    win_buf[2][2][ic] = val2;
}

if(r >= 2 && c >= 2) {
    for(oc = 0; oc < C1; oc++) {
#pragma HLS PIPELINE II=1
        sum = bias[oc];

        for(kr = 0; kr < 3; kr++) {
///#pragma HLS UNROLL
            for(kc = 0; kc < 3; kc++) {
///#pragma HLS UNROLL
                for(ic = 0; ic < 3; ic++) {
                    sum += win_buf[kr][kc][ic] * kernel[oc][kr][kc][ic];
                }
            }
        }
    }
}

```

```

        if(sum < 0) sum = 0.0f;
        out[r-2][c-2][oc] = sum;
    }
}
}
}
}

/* =====

===== */

void maxpool1(
    float in[62][62][C1],
    float out[31][31][C1])
{
    int r, c, ch;
    float v0, v1, v2, v3, maxv;

    for(r = 0; r < 31; r++) {
        for(c = 0; c < 31; c++) {
#pragma HLS UNROLL
            for(ch = 0; ch < C1; ch++) {

                v0 = in[2*r][2*c][ch];
                v1 = in[2*r][2*c+1][ch];
                v2 = in[2*r+1][2*c][ch];
                v3 = in[2*r+1][2*c+1][ch];

                maxv = v0;
                if(v1 > maxv) maxv = v1;

```

```

        if(v2 > maxv) maxv = v2;
        if(v3 > maxv) maxv = v3;

        out[r][c][ch] = maxv;
    }
}
}
}

/* =====

===== */

void conv2(
    float in[31][31][C1],
    float out[29][29][C2],
    float kernel[C2][C1][3][3],
    float bias[C2])
{
    static float linbu[2][31][C1];
    float win_buf[3][3][C1];
    int r, c, ic, oc, kr, kc;
    float val0, val1, val2, sum;
#pragma HLS ARRAY_PARTITION variable=linbu complete dim=1
    #pragma HLS ARRAY_PARTITION variable=linbu cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=win_buf complete dim=1
    #pragma HLS ARRAY_PARTITION variable=win_buf complete dim=2
    #pragma HLS ARRAY_PARTITION variable=win_buf cyclic factor=8 dim=3
    for(r = 0; r < 31; r++) {
        for(c = 0; c < 31; c++) {
            for(ic = 0; ic < C1; ic++) {

```

```

#pragma HLS UNROLL

    win_buf[0][0][ic] = win_buf[0][1][ic];
    win_buf[0][1][ic] = win_buf[0][2][ic];
    win_buf[1][0][ic] = win_buf[1][1][ic];
    win_buf[1][1][ic] = win_buf[1][2][ic];
    win_buf[2][0][ic] = win_buf[2][1][ic];
    win_buf[2][1][ic] = win_buf[2][2][ic];

    val0 = linbu[0][c][ic];
    val1 = linbu[1][c][ic];
    val2 = in[r][c][ic];

    linbu[0][c][ic] = val1;
    linbu[1][c][ic] = val2;

    win_buf[0][2][ic] = val0;
    win_buf[1][2][ic] = val1;
    win_buf[2][2][ic] = val2;
}

if(r >= 2 && c >= 2) {

    for(oc = 0; oc < C2; oc++) {
#pragma HLS PIPELINE II=1
        sum = bias[oc];
        for(kr = 0; kr < 3; kr++) {
            ##pragma HLS UNROLL
                for(kc = 0; kc < 3; kc++) {
            ##pragma HLS UNROLL
                for(ic = 0; ic < C1; ic++) {

```



```

        v3 = in[2*r+1][2*c+1][ch];

        maxv = v0;
        if(v1 > maxv) maxv = v1;
        if(v2 > maxv) maxv = v2;
        if(v3 > maxv) maxv = v3;

        out[r][c][ch] = maxv;
    }
}
}
}

/* =====

===== */

void conv3(
    float in[14][14][C2],
    float out[12][12][C3],
    float kernel[C3][C2][3][3],
    float bias[C3])
{
    static float linbu[2][14][C2];
    float win_buf[3][3][C2];
    int r, c, ic, oc, kr, kc;
    float val0, val1, val2, sum;
#pragma HLS ARRAY_PARTITION variable=linbu complete dim=1
    #pragma HLS ARRAY_PARTITION variable=linbu cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=win_buf complete dim=1
    #pragma HLS ARRAY_PARTITION variable=win_buf complete dim=2

```

```

#pragma HLS ARRAY_PARTITION variable=win_buf cyclic factor=8 dim=3
for(r = 0; r < 14; r++) {
    for(c = 0; c < 14; c++) {
        for(ic = 0; ic < C2; ic++) {
#pragma HLS UNROLL
            win_buf[0][0][ic] = win_buf[0][1][ic];
            win_buf[0][1][ic] = win_buf[0][2][ic];
            win_buf[1][0][ic] = win_buf[1][1][ic];
            win_buf[1][1][ic] = win_buf[1][2][ic];
            win_buf[2][0][ic] = win_buf[2][1][ic];
            win_buf[2][1][ic] = win_buf[2][2][ic];

            val0 = linbu[0][c][ic];
            val1 = linbu[1][c][ic];
            val2 = in[r][c][ic];

            linbu[0][c][ic] = val1;
            linbu[1][c][ic] = val2;

            win_buf[0][2][ic] = val0;
            win_buf[1][2][ic] = val1;
            win_buf[2][2][ic] = val2;
        }

        if(r >= 2 && c >= 2) {
            for(oc = 0; oc < C3; oc++) {
#pragma HLS PIPELINE II=1
                sum = bias[oc];
                for(kr = 0; kr < 3; kr++) {
                    ##pragma HLS UNROLL

```

```

        for(kc = 0; kc < 3; kc++) {
//#pragma HLS UNROLL
            for(ic = 0; ic < C2; ic++) {
                sum += win_buf[kr][kc][ic] * kernel[oc][ic][kr][kc];
            }
        }
    }
    if(sum < 0) sum = 0.0f;
    out[r-2][c-2][oc] = sum;
}
}
}
}

/* =====

===== */

void conv4(
    float in[12][12][C3],
    float out[10][10][C3],
    float kernel[C3][C3][3][3],
    float bias[C3])
{
    static float linbu[2][12][C3];
    float win_buf[3][3][C3];
    int r, c, ic, oc, kr, kc;
    float val0, val1, val2, sum;
#pragma HLS ARRAY_PARTITION variable=linbu complete dim=1
    #pragma HLS ARRAY_PARTITION variable=linbu cyclic factor=8 dim=3

```

```

#pragma HLS ARRAY_PARTITION variable=win_buf complete dim=1
#pragma HLS ARRAY_PARTITION variable=win_buf complete dim=2
#pragma HLS ARRAY_PARTITION variable=win_buf cyclic factor=8 dim=3
for(r = 0; r < 12; r++) {
    for(c = 0; c < 12; c++) {
        for(ic = 0; ic < C3; ic++) {
#pragma HLS UNROLL
            win_buf[0][0][ic] = win_buf[0][1][ic];
            win_buf[0][1][ic] = win_buf[0][2][ic];
            win_buf[1][0][ic] = win_buf[1][1][ic];
            win_buf[1][1][ic] = win_buf[1][2][ic];
            win_buf[2][0][ic] = win_buf[2][1][ic];
            win_buf[2][1][ic] = win_buf[2][2][ic];

            val0 = linbu[0][c][ic];
            val1 = linbu[1][c][ic];
            val2 = in[r][c][ic];

            linbu[0][c][ic] = val1;
            linbu[1][c][ic] = val2;

            win_buf[0][2][ic] = val0;
            win_buf[1][2][ic] = val1;
            win_buf[2][2][ic] = val2;
        }

        if(r >= 2 && c >= 2) {
            for(oc = 0; oc < C3; oc++) {
#pragma HLS PIPELINE II=1
                sum = bias[oc];
            }
        }
    }
}

```

```

        for(kr = 0; kr < 3; kr++) {
//#pragma HLS UNROLL
            for(kc = 0; kc < 3; kc++) {
//#pragma HLS UNROLL
                for(ic = 0; ic < C3; ic++) {
                    sum += win_buf[kr][kc][ic] * kernel[oc][ic][kr][kc];
                }
            }
        }
        if(sum < 0) sum = 0.0f;
        out[r-2][c-2][oc] = sum;
    }
}
}
}
}
}

```

```

/* =====
===== */

```

```

void conv5(
    float in[10][10][C3],
    float out[8][8][C3],
    float kernel[C3][C3][3][3],
    float bias[C3])
{
    static float linbu[2][10][C3];
    float win_buf[3][3][C3];
    int r, c, ic, oc, kr, kc;
    float val0, val1, val2, sum;

```

```

#pragma HLS ARRAY_PARTITION variable=linbu complete dim=1
    #pragma HLS ARRAY_PARTITION variable=linbu cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=win_buf complete dim=1
    #pragma HLS ARRAY_PARTITION variable=win_buf complete dim=2
    #pragma HLS ARRAY_PARTITION variable=win_buf cyclic factor=8 dim=3
    for(r = 0; r < 10; r++) {
        for(c = 0; c < 10; c++) {
            for(ic = 0; ic < C3; ic++) {
#pragma HLS UNROLL
                win_buf[0][0][ic] = win_buf[0][1][ic];
                win_buf[0][1][ic] = win_buf[0][2][ic];
                win_buf[1][0][ic] = win_buf[1][1][ic];
                win_buf[1][1][ic] = win_buf[1][2][ic];
                win_buf[2][0][ic] = win_buf[2][1][ic];
                win_buf[2][1][ic] = win_buf[2][2][ic];

                val0 = linbu[0][c][ic];
                val1 = linbu[1][c][ic];
                val2 = in[r][c][ic];

                linbu[0][c][ic] = val1;
                linbu[1][c][ic] = val2;

                win_buf[0][2][ic] = val0;
                win_buf[1][2][ic] = val1;
                win_buf[2][2][ic] = val2;
            }

            if(r >= 2 && c >= 2) {
                for(oc = 0; oc < C3; oc++) {

```

```

#pragma HLS PIPELINE II=1

    sum = bias[oc];
    for(kr = 0; kr < 3; kr++) {
//#pragma HLS UNROLL
        for(kc = 0; kc < 3; kc++) {
//#pragma HLS UNROLL
            for(ic = 0; ic < C3; ic++) {
                sum += win_buf[kr][kc][ic] * kernel[oc][ic][kr][kc];
            }
        }
    }

    out[r-2][c-2][oc] = sum;
}
}
}
}
}

/* =====

===== */

float sigmoid(float x)
{
#pragma HLS INLINE
    return 1.0f / (1.0f + expf(-x));
}

/* =====

```

```

===== */
void detection_head(
    float in[8][8][C3],
    float out[8][8][OUT_CH],
    float kernel[OUT_CH][C3][1][1],
    float bias[OUT_CH])
{
    int r, c, oc, ic;

    float sum[OUT_CH];
#pragma HLS ARRAY_PARTITION variable=sum complete

    for(r = 0; r < 8; r++) {
        for(c = 0; c < 8; c++) {

            for(oc = 0; oc < OUT_CH; oc++) {

                sum[oc] = bias[oc];
            }

            for(ic = 0; ic < C3; ic++) {
#pragma HLS PIPELINE II=1
                float val = in[r][c][ic];

                for(oc = 0; oc < OUT_CH; oc++) {

                    sum[oc] += val * kernel[oc][ic][0][0];
                }
            }
        }
    }
}

```



```

    }

    for(oc = 0; oc < OUT_CH; oc++) {
        out[r][c][oc] = sigmoid(sum[oc]);
    }
}
}
}

/* =====

===== */

void topp(
    float image[64][64][3],
    float output[8][8][OUT_CH]
)
{
#pragma HLS DATAFLOW
    static float f1[62][62][C1];
    static float p1[31][31][C1];
    static float f2[29][29][C2];
    static float p2[14][14][C2];
    static float f3[12][12][C3];
    static float f4[10][10][C3];
    static float f5[8][8][C3];

#pragma HLS ARRAY_PARTITION variable=f1 cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=p1 cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=f2 cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=p2 cyclic factor=8 dim=3

```

```

#pragma HLS ARRAY_PARTITION variable=f3 cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=f4 cyclic factor=8 dim=3
#pragma HLS ARRAY_PARTITION variable=f5 cyclic factor=8 dim=3

conv1(image, f1, W1, B1);
maxpool1(f1, p1);
conv2(p1, f2, W2, B2);
maxpool2(f2, p2);
conv3(p2, f3, W3, B3);
conv4(f3, f4, W4, B4);
conv5(f4, f5, W5, B5);
detection_head(f5, output, WH, BH);
}

```

Code 4: thuật toán TinyYolo-lite cải tiến

Dẫn hướng #pragma HLS DATAFLOW được gắn ở hàm topp kích hoạt tính năng Task-level Pipelining. Thay vì thực thi tuần tự (chạy xong toàn bộ conv1 rồi mới đến maxpool1, xong maxpool1 mới đến conv2), DATAFLOW cho phép các hàm này chạy song song. Ngay khi conv1 tính toán xong một phần dữ liệu, maxpool1 có thể bắt đầu xử lý ngay phần dữ liệu đó. Việc này giúp giảm độ trễ tổng thể một cách đột phá. Thay vì tổng thời gian bằng tổng thời gian của từng layer cộng lại, thời gian chạy sẽ phụ thuộc vào layer chậm nhất (bottleneck).

=> Để các hàm có thể truyền dữ liệu cho nhau một cách song song, HLS compiler sẽ tự động chèn các bộ đệm FIFOs vào giữa các hàm khiến FIFOs tăng vọt lên

Dẫn hướng #pragma HLS ARRAY_PARTITION được áp dụng cho các mảng như linbu (Line buffer), win_buf (Window buffer), và các feature maps chia các mảng thành các thanh ghi độc lập. Việc chia nhỏ các mảng giúp giải quyết được vấn đề cấu trúc BRAM tiêu chuẩn trên FPGA chỉ có 2 cổng đọc/ghi (Dual-port RAM). Nếu một vòng lặp muốn lấy 9 giá trị cùng lúc (cho một kernel 3x3), BRAM sẽ báo lỗi thắt cổ chai (memory bottleneck). Lệnh này "chẻ" bộ nhớ ra làm nhiều phần để cho phép truy cập song song.

=> Để các mảng chia nhỏ ra thành các thanh ghi, các Flip-Flops và LUTs sẽ được sử dụng

Dẫn hướng #pragma HLS PIPELINE II=1 được gắn tại các vòng lặp tính toán nhân chập và Detection Head. Kích hoạt Loop-level Pipelining. II=1 (Initiation Interval = 1) nghĩa là compiler được yêu cầu cứ sau 1 xung nhịp đồng hồ (clock cycle) phải đưa được một vòng lặp mới vào đường ống xử lý nếu có thể, không cần chờ vòng lặp trước đó tính toán xong.

=> Để có thể thực hiện pipeline, compiler phải tạo ra nhiều phần cứng để đáp ứng việc dữ liệu chảy liên tục, các LUT và FIFO sẽ được tăng cường sử dụng để lưu trữ các trạng thái trung gian trong đường ống.

Dẫn hướng #pragma HLS UNROLL giúp trải phẳng vòng lặp, tức thay vì dùng 1 khối phần cứng thực hiện N lần lặp lại, nó sẽ tạo ra N khối phần cứng độc lập để thực hiện tất cả trong 1 chu kỳ duy nhất giúp triệt tiêu hoàn toàn độ trễ của việc quản lý vòng lặp. Đưa thời gian xử lý của đoạn code về mức song song tối đa.

=> Số lượng LUTs và FFs sẽ tăng theo cấp số nhân với số lần unroll

Dẫn hướng #pragma HLS INLINE yêu cầu compiler không tạo một module phần cứng riêng biệt cho hàm này. Thay vào đó, nó sẽ bê nguyên logic phần cứng của hàm sigmoid vào nơi gọi nó (trong hàm detection_head). Việc này giúp loại bỏ độ trễ của việc gọi hàm (function call overhead) như thao tác thiết lập handshake, truyền tham số, và nhận kết quả.

=> Do hàm sigmoid chỉ được gọi tại một vòng lặp trong detection_head, nên chi phí phần cứng không bị phình quá to mà chỉ sử dụng tài nguyên DSPs và LUTs cần thiết để tính toán hàm mũ.

6. Mô phỏng/thực thi và đánh giá

6.1. Kết quả synthesis

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty	
ap_clk	10.00	8.677	1.25	

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
31110	57345	31042	57346	dataflow

Detail

Instance

Instance	Module	Latency		Interval		Type
		min	max	min	max	
maxpool1_U0	maxpool1	23809	23809	23809	23809	none
detection_head_U0	detection_head	31041	31041	31041	31041	none
maxpool2_U0	maxpool2	9395	9395	9395	9395	none
conv1_U0	conv1	20481	57345	20481	57345	none
conv4_U0	conv4	721	5473	721	5473	none
conv5_U0	conv5	501	3801	501	3801	none
conv3_U0	conv3	589	7057	589	7057	none
conv2_U0	conv2	1923	18260	1923	18260	none

Log

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	-	-	-
Expression	-	-	0	2
FIFO	0	-	392	2744
Instance	3	17	11109	12138
Memory	-	-	-	-
Multiplexer	-	-	-	-
Register	-	-	-	-
Total	3	17	11501	14884
Available	280	220	106400	53200
Utilization (%)	1	7	10	27

Hình 12: Kết quả synthesis mô hình suy luận tối ưu

Nhận xét:

- + Kết quả synthesis đã thỏa mãn được ràng buộc đặt ra cho xung clk
- + Latency đã giảm từ 48873231 xuống còn trong khoảng từ 31110 đến 57345
- + Interval đã giảm từ 48873231 xuống còn trong khoảng từ 31110 đến 57346

+ Việc sử dụng nhiều LUT có thể giải thích do dùng nhiều dẫn hướng array_partition, unroll, pipeline

6.2. Kết quả C/RTL simulation

Cosimulation Report for 'topp'

Result							
		Latency			Interval		
RTL	Status	min	avg	max	min	avg	max
VHDL	Pass	37712	37712	37712	NA	NA	NA
Verilog	NA	NA	NA	NA	NA	NA	NA

Export the report(.html) using the [Export Wizard](#)

Hình 13: Kết quả C/RTL simulation

7. Kết quả triển khai phần cứng trên ZynQ

Export Report for 'topp'

General Information

Report date: Mon Jun 08 16:00:12 +0700 2026
Project: topp
Solution: solution1
Device target: xc7z020clg484-1
Implementation tool: Xilinx Vivado v.2018.3

Resource Usage

	VHDL
SLICE	1772
LUT	3832
FF	4241
DSP	17
BRAM	0
SRL	27

Final Timing

	VHDL
CP required	10.000
CP achieved post-synthesis	8.337
CP achieved post-implementation	8.954

Timing met

Hình 14: Kết quả triển khai phần cứng

8. Kết luận

Báo cáo đã trình bày chi tiết về phương pháp xây dựng thuật toán mô phỏng mô hình suy luận Tiny Yolo lite sử dụng Vivado HLS. Báo cáo cũng nêu ra được phương pháp cải tiến và so sánh giữa các phương pháp với việc đánh đổi phần cứng để tăng tốc xử lý. Việc triển khai trên mạch phần cứng ZYNQ-7 ZC702 Evaluation Board cũng là khả thi với kết quả mô phỏng đạt được.

List of Figures

Hình 1. Cấu trúc phân lớp của mô hình TinyYoLo-lite.	5
Hình 2. Ví dụ minh họa phép tính tích chập	7
Hình 3. Ví dụ minh họa max pooling.....	8
Hình 4: Ví dụ minh họa detection head.....	9
Hình 5: Đầu ra channel 0 test bench	28
Hình 6: Đầu ra bounding box + class score test bench	29
Hình 7: Kết quả synthesis mô hình suy luận cơ bản	30
Hình 8: Mô tả cách hoạt động của dẫn hướng array_partition.....	32
Hình 9: Mô tả cách hoạt động của dẫn hướng unroll.....	32
Hình 10: Mô tả cách hoạt động của dẫn hướng pipeline.....	33
Hình 11: Mô tả cách hoạt động của dẫn hướng dataflow.....	33
Hình 12: Kết quả synthesis mô hình suy luận tối ưu	51
Hình 13: Kết quả C/RTL simulation.....	52
Hình 14: Kết quả triển khai phần cứng	53

List of Code

Code 1: file nguồn thuật toán	9
Code 2: thuật toán TinyYolo-lite cơ bản.....	20
Code 3: testbench thuật toán TinyYolo-lite cơ bản.....	28
Code 4: thuật toán TinyYolo-lite cải tiến.....	50

References

- [1] <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>
- [2] https://xilinx.github.io/Vitis-Tutorials/2021-1/build/html/docs/Hardware_Acceleration/Design_Tutorials/01-convolution-tutorial/lab2_conv_filter_kernel_design.html
- [3] Nguyen Kiem Hung, T. Q. (2023). A Bandwidth-Efficient High-Performance RTL Microarchitecture of 2D-Convolution for Deep Neural Networks. VNU Journal of Science: Computer Science and Communication Engineering.
- [4] <https://docs.amd.com/r/en-US/ug1399-vitis-hls/pragma-HLS-dataflow>